CREDIT RISK ANALYTICS

Bank Loan Risk Analysis Platform

From raw credit-card data to explainable risk scores, an API and an analyst dashboard

BY:
YUVASREE R

Problem statement

Risk hides in history

Six months of repayment status, bills and payments must be read together. A single late month or a high balance alone tells little.

Raw data is messy

Education and marriage codes fall outside the documented set, and some bill amounts are negative (credit balances).

Decisions need a trail

Reviewers must record status, priority and notes for each flagged customer, and data loads must be auditable.

Dataset: 30,000 credit-card customers, 6,636 (22.1%) defaulted the following month.

Business need

Spot high-risk customers early

Rank all 30,000 accounts by a 0 to 100 score and Low, Medium or High band, so reviewers start with the riskiest.

Trust the numbers

Every load is validated and reconciled. Rejected records are exported with their reason and audited.

Explain every score

Each rating stores per-indicator points, a rule version and a plain-language reason.

Control access

Analysts and administrators sign in with JWT tokens. Only administrators can delete reviews.

Solution overview

Data pipeline

Ten-step ETL from Excel to MySQL: validation, transformation, incremental classification, batch loading, reconciliation and archiving.

Risk engine

Configurable rules in risk_config.py turn four indicators into a score and band. It never modifies the database itself.

Secure API

FastAPI routers for dashboard, customers, reviews, exports and auth, with Pydantic response schemas.

Analyst workspace

Streamlit app with Dashboard, Customer Explorer, Risk Reviews and Data Export pages.

Architecture

1. Source

Excel (.xls or .xlsx), sheet Data, dropped into the incoming folder.

2. ETL

extract, data_quality, transform, incremental, load and reconciliation modules.

3. MySQL

Core tables, ETL audit tables, aggregate tables and risk tables.

4. Risk engine

Reads credit profiles, scores each customer, saves to customer_risk_scores.

5. FastAPI

Auth, dashboard, customers, reviews and exports, plus health and database checks.

6. Streamlit

Four pages calling the API, with analyst and administrator views.

The dataset and derived metrics

30,000
customers
One row per credit-card client

22.1%
defaulted next month
6,636 customers in the data

25
raw columns
Per customer in the Excel sheet

RAW COLUMN GROUPS

Profile
Credit limit, sex, education, marriage status and age.

Repayment
Repayment status for each of the six months.

Bills
Bill amount for each of the six months.

Payments
Amount paid in each of the six months.

Target
Default next month: 1 means default, 0 means no default.

DERIVED METRICS

Utilization
September bill ÷ credit limit

Payment-to-bill
Total payments ÷ total bills

Months with delay
Months with status above 0

Maximum delay
Worst monthly status

Data flow: ten-step ETL pipeline

1. Extract source file
2. Validate data quality
3. Transform and derive metrics
4. Classify new, duplicate, corrected
5. Create ETL run record
6. Record quality issues
7. Export curated snapshots
8. Load MySQL in batches
9. Reconcile source and database
10. Archive the source file

Data quality

Checks for missing values, age, credit limit and repayment codes. Rejected rows are exported and audited.

Incremental loads

Record hashes detect new, duplicate and corrected customers.

Reconciliation

Every run is compared with the database and logged with an ETL lock.

Database design

Core data · 4 tables

customers (profile, labels, age group, record hash), credit_profiles (limit, six repayment statuses, delay metrics, totals), bills and payments (180,000 rows each).

ETL audit · 3 tables

etl_runs (counts and reconciliation status per run), etl_record_audit (action and reason per record), data_quality_issues (check, column, invalid value).

Aggregates · 4 tables

agg_portfolio_summary, agg_risk_distribution, agg_monthly_financials and agg_segment_risk. Refreshed by a script; the dashboard reads them.

Risk · 2 tables

customer_risk_scores (score, band, four component scores, rule version, reason) and customer_risk_reviews (status, priority, notes, review date).

Bills, payments, scores and reviews link to customers through customer_id foreign keys.

Rule-based, explainable risk scoring

Indicator	Medium: 10 points	High: 25 points
Max repayment delay	1 month	2 months or more
Months with delay	1 to 2	3 or more
Credit utilization	30% to 79%	80% or more
Payment-to-bill ratio	30% to 79%	Below 30%

Low risk

Score 0 to 29

Medium risk

Score 30 to 59

High risk

Score 60 to 100

Four indicators, up to 25 points each. Scores are configurable heuristics (rule version 1.0), not calibrated default probabilities.

API endpoints and access control

Router	Endpoints	Access
Auth	POST /api/auth/login issues a JWT (30 minutes). GET /api/auth/me returns the signed-in user.	Login is open; every other router needs a valid token
Dashboard	GET /api/dashboard/portfolio, /risk-distribution, /monthly-financials and /segment-risk, read from the aggregate tables.	Analyst or administrator
Customers	GET /api/customers is paginated (20 per page, max 100) with age and credit-limit filters. GET /api/customers/{customer_id} returns one profile.	Analyst or administrator
Reviews	POST creates a review. GET lists them (paginated, filter by customer_id). GET and PATCH /api/reviews/{review_id} read and update one.	Analyst or administrator
Reviews: delete	DELETE /api/reviews/{review_id} removes a review.	Administrator only
Exports	GET /api/exports/customers streams a CSV using the same age and credit-limit filters.	Analyst or administrator
Health	GET /health and /health/db check the API and the database. Interactive docs at /docs.	Open

Technology stack

Data and ETL

Python, pandas and NumPy read Excel, validate, derive metrics and write curated snapshots.

Database

MySQL accessed through SQLAlchemy and PyMySQL, with indexes and aggregate tables.

Backend

FastAPI with Pydantic models and PyJWT authentication.

Frontend

Streamlit pages for the dashboard, explorer, reviews and exports.

Configuration

python-dotenv for secrets and a versioned risk-rule config.

Testing

pytest, with text and XML reports for every suite.

Demo walkthrough

1. Sign in

Log in as analyst or administrator; a 30-minute token is issued.

2. Dashboard

Portfolio summary, risk distribution, monthly financials and segment risk.

3. Customer Explorer

Find a customer and see the score, band and reasons.

4. Reviews and export

Open a review, set status and priority, then export customer data.

Evaluation results

293

tests passed

Full suite in 32.47 seconds

11

test suites

ETL, risk rules, API, frontend

30,000

valid records

No rejects in notebook profiling

Checks include relational integrity (six bill and six payment rows per customer), reconciliation after each load, and API and frontend integration.

Scores are heuristics and were not validated against actual defaults, so no predictive accuracy is claimed.

What the tests cover

- Extract
- Data quality
- Transform
- Incremental load
- Load
- Pipeline
- Reconciliation
- Reporting aggregates
- Business rules
- API
- Frontend integration

Each suite writes a text and XML report to the test_reports folder.

Challenges and learning

Challenge	How it was handled
Negative utilization	Treated as zero utilization
Zero or negative net bill	No payment-ratio points assigned
Incomplete records	Policy is UNCLASSIFIED, never LOW
Repeated loads	Record hashes classify new, duplicate, corrected
Invalid source rows	Rejected with reasons, exported and audited
Overlapping runs	ETL lock, released after each run

Learning: keep rules in versioned config, and test every layer.

Summary and next steps

What was built

Validated, reconciled ETL; explainable rule-based scores; a role-secured API; an analyst dashboard; 293 passing tests.

Suggested next steps

Validate scores against actual defaults, calibrate weights, add trend alerts, and schedule the pipeline.

THANK YOU